MIPS 指令集归纳整理

目录

指令集

- （壹）整数指令集
  - 一、数据传送类指令
  - 二、算数运算类指令
  - 三、逻辑运算类指令
  - 四、移位指令
  - 五、逻辑设置指令
  - 六、跳转指令
  - 七、系统调用

宏指令

- （贰）宏指令（伪指令）

寄存器

- MIPS 通用寄存器表（32 个）
- 特殊寄存器（非通用寄存器）

问答

- 第一部分：概述与 MIPS 架构
- 第二部分：寻址方式与指令

附录

- MARS 常用 SYSCALL 服务表

前情:

$!!!😒 **逻辑运算 0 扩展；其余 I 型指令一律符号扩展。u 不改变扩展方式，只改变运算时的溢出检测和数值解读方式(即最高位是作为符号位还是作为数值位)。**

注意：

rd:register destination(一般用于在R指令格式中作目的寄存器)

rs:register source(I、R指令格式中的第一源寄存器)

rt:register target(I指令格式中的目的寄存器、R指令格式中的第二源寄存器)

PC的问题:

一、问题的核心

我们需要的结果是：

- `jal` 保存到 `$ra` 的值 = **当前 `jal` 指令的地址 + 4**

但在 CPU 内部：

- 当 `jal` 进入**执行阶段**时，**PC 已经在取指阶段结束时就变成了** `PC + 4`（指向下一条指令）

那么问题来了：

> **执行阶段的 `jal` 怎么还能拿到"原 PC"来算 `原 PC + 4`？**

二、两种硬件实现方案（没有"可能"，是确定的设计选择）

方案 A：保存取指开始时的 PC 值

- 在取指阶段开始时，把当前的 PC 值复制一份到一个**隐藏的临时寄存器**（比如 `temp_PC`）
- 取指结束后，PC 更新为 `PC + 4`
- 执行阶段，`jal` 指令直接用 `temp_PC + 4` 写入 `$ra`

方案 B：执行阶段用 `PC - 4` 恢复原 PC

- 执行阶段时，当前 PC 已经是 `原 PC + 4`
- `jal` 指令内部做一次减法：`(PC - 4) + 4`，等价于 `原 PC + 4`

三、不同 MIPS 实现的选择

实现	方案	特点
经典 MIPS 单周期	方案 A（保存原 PC）	电路清晰，但多一个寄存器
经典 MIPS 流水线	方案 B（PC - 4）	不增加额外寄存器，但需要减法器
教学简化模型	通常用方案 A	更直观，学生易理解

所以"可能"不是指随机，而是不同教材/不同硬件实现可能选不同方案。

四、正确的说法（没有"可能"）

> **为了保证 `jal` 能正确保存返回地址，硬件必须能够获得当前指令的原始 PC 值。**
> 具体实现方式有两种：

> 1. 在取指阶段保存原 PC 到临时寄存器
> 2. 在执行阶段用当前 PC 减去 4 恢复原 PC
>
> **两种方案都能正确工作，结果相同。**

五、总结

你抓到的“可能”确实是我的表述不够精确。
更准确的说法是：**有两种确定的硬件实现方式，不同 MIPS 处理器会选择其中一种。**

感谢你的追问，这让回答更严谨了。

（壹）整数指令集

一、数据传送类指令

1. 取（加载）数据指令

- `lb rt, offset(rs)` # load byte [I]
  `rs` 的值加上**符号扩展**的 16 位偏移量形成有效地址，从有效地址指向的内存读取 **8 位字节**，**符号扩展**后加载到 `rt`。

- `lbu rt, offset(rs)` # load byte unsigned [I]
  `rs` 的值加上**符号扩展**的 16 位偏移量形成有效地址，从有效地址指向的内存读取 **8 位字节**，**0扩展**后加载到 `rt`。

- `lh rt, offset(rs)` # load halfword [I]
  `rs` 的值加上**符号扩展**的 16 位偏移量形成有效地址，从有效地址指向的内存读取 **16 位半字**，**符号扩展**后加载到 `rt`。

  > 若有效地址为奇数则发生地址错误异常，地址必为 2 的倍数（0 bit 为 0）。

- `lhu rt, offset(rs)` # load halfword unsigned [I]
  `rs` 的值加上**符号扩展**的 16 位偏移量形成有效地址，从有效地址指向的内存读取 **16 位半字**，**0扩展**后加载到 `rt`。

  > 若有效地址为奇数则发生地址错误异常，地址必为 2 的倍数（0 bit 为 0）。

- `lw rt, offset(rs)` # load word [I]
  `rs` 的值加上**符号扩展**的 16 位偏移量形成有效地址，从有效地址指向的内存读取 **字数据** 加载到 `rt`。

  > 地址应为 4 的倍数（末 2 bit 为 0），否则发生地址错误异常。

- `lui rt, imm` # load upper immediate [I]加载立即数到高半字（16bits）
  `rt = imm << 16 | 0x0000`

将 16 位立即数左移 16 位，低 16 位补 0，结果存入 `rt`。

2. 存数据指令

- `sb rt, offset(rs) # store byte [I]`
  有效地址 = `rs` + 【符号扩展】的 16 位偏移量，将 `rt` 最低 8 位存入有效地址指向的内存。
- `sh rt, offset(rs) # store halfword [I]`
  有效地址 = `rs` + 【符号扩展】的 16 位偏移量，将 `rt` 低 16 位存入有效地址指向的内存（地址必为 2 的倍数）。
- `sw rt, offset(rs) # store word [I]`
  有效地址 = `rs` + 【符号扩展】的 16 位偏移量，将 `rt` 全部 32 位存入有效地址指向的内存（地址必为 4 的倍数）。

四、总结

问题	答案
写入时数据按有符号还是无符号?	**都不是**，只取低 8 位，纯进制
内存里的字节有符号属性吗?	**没有**，只有 0/1
谁决定这个字节是有符号还是无符号?	**读取指令**（lb 或 lbu）
sb 配合 lbu 能正确读回无符号数吗?	✅ 能，因为 sb 只存低 8 位，lbu 零扩展
sb 配合 lb 能正确读回有符号数吗?	✅ 能，因为 lb 会符号扩展

一句话：内存只存二进制位，不存符号属性。符号是你读的时候加上的解释。

3. 专用寄存器

- `mfhi rd # move from HI register[R]`
- `mflo rd # move from LO register[R]`
- `mthi rs # move to HI register[R]`
- `mtlo rs # move to LO register[R]`

二、算数运算类指令

1. 加

- `add rd, rs, rt` [R]
  `rs + rt` → `rd`，可能检查补码溢出。
- `addu rd, rs, rt` [R]
  不检查补码溢出（一般用于地址运算）。
- `addi rt, rs, imm` [I]
  `rs` + **符号扩展**至 32 位的立即数 → `rt`，会检查补码溢出。
- `addiu rt, rs, imm` [I]
  `rs` + **符号扩展**至 32 位的立即数 → `rt`，不检查补码溢出。

  > **注**：`unsigned` 仅代表不检查补码溢出，**不代表零扩展**。立即数仍进行【符号扩展】，只是将最高位视为数值位。

"检查"这个词很微妙

> addiu 和 addi 在立即数扩展上完全相同（都是符号扩展），唯一的区别是 addiu 不检查溢出，把结果当作无符号数处理。

2. 减

- `sub rd, rs, rt` [R]
  `rs - rt` → `rd`，会检查补码溢出。
- `subu rd, rs, rt` [R]
  不检查补码溢出。

3. 乘

- `mult rs, rt` [R]
  `rs * rt`，64 位积：高 32 位 → `HI`，低 32 位 → `LO`。操作数视为有符号数，不产生溢出。
  结果可用 `mfhi` / `mflo` 取出。
- `multu rs, rt` [R]
  操作数视为无符号正数，其余同 `mult`。

4. 除

- `div rs, rt` [R]
  `rs ÷ rt`：商 → `LO`，余数 → `HI`。操作数符号相反时商为负；余数符号与被除数 `rs` 相同。不产生溢出异常。

  > 除数为 0 时结果未定义。

- `divu rs, rt` [R]
  操作数均视为无符号数，商和余数恒为正。其余同 `div`。

三、逻辑运算类指令

- `and rd, rs, rt` [R]
  rs与rt按位与 → `rd`。

- `andi rt, rs, imm` [I]
  `rs` 与 **0扩展**至 32 位的立即数按位与 → `rt`。

  > 注：立即数高 16 位默认为 0，若需非 0 掩码，先用 `lui` + `ori` 构造。

- `or rd, rs, rt` [R]
  按位或 → `rd`。

- `ori rt, rs, imm` [I]
  `rs` 与 **0扩展**至 32 位的立即数按位或 → `rt`。

- `nor rd, rs, rt` [R]
  按位或非（先或后取反） → `rd`。

- `xor rd, rs, rt` [R]
  按位异或 → `rd`。

- `xori rt, rs, imm` [I]
  `rs` 与 **0扩展**的立即数按位异或 → `rt`。

四、移位指令

- `sll rd, rt, sa shift left logical`[R]
  `rt` 左移 `sa` 位，空位补 【0】 → `rd`。

- `sllv rd, rt, rs shift left logical variable`[R]
  `rt` 左移 `rs` 低 5 位指定的位数，空位补 【0】 → `rd`。

  > 最多移位 31 位（5 bit 全 1）。

- `srl rd, rt, sa shift right logical`[R]
  `rt` 逻辑右移 `sa` 位，空位补 【0】 → `rd`。

- `srlv rd, rt, rs shift right logical variable`[R]
  `rt` 逻辑右移 `rs` 低 5 位指定的位数，空位补 【0】 → `rd`。

- `sra rd, rt, sa` `shift right arithmetic[R]`
  `rt` 算术右移 `sa` 位，空位用【符号位】填充 → `rd`。
- `srav rd, rt, rs` `shift right arithmetic variable[R]`
  `rt` 算术右移 `rs` 低 5 位指定的位数，空位用【符号位】填充 → `rd`。

五、逻辑设置指令

- `slt rd, rs, rt` `set if less than[R]`
  若 `rs < rt`（有符号比较），`rd` 置 1，否则置 0。
- `sltu rd, rs, rt` `set if less than unsigned[R]`
  若 `rs < rt`（无符号比较），`rd` 置 1，否则置 0。
- `slti rt, rs, imm` `set if less than immediate[I]`
  若 `rs < 【符号扩展】的立即数`（有符号），`rt` 置 1，否则置 0。
- `sltiu rt, rs, imm` `set if less than immediate unsigned[I]`
  若 `rs < 【符号扩展】的立即数`（无符号），`rt` 置 1，否则置 0。

六、跳转指令

1. 无条件转移指令

- `j label` `[J]`
  PC 高 4 位拼接 26 位立即数左移 2 位 → PC。

  > 左移两位-->指令均为32位，以字为单位，故0、1bit必为0
  > 指令的增加，一般不会影响到高四位

- `jal label` `jump and link[J]`

  > 将PC当前值保存在$ra中，然后将【当前PC的前四位】与【26位立即数左移两位之后】连接起来形成一个地址，加载到PC。

- `jalr rd, rs` `jump and link register[R]`
  将返回地址（PC+4）存入 `rd`，然后跳转至 `rs` 中的地址。

  > 使用前 `rs` 必须已加载有效地址。

- `jr rs` `jump register unconditionally[R]`
  跳转至 `rs` 中的地址。常用于函数返回：`jr $ra`。

2. 分支指令

- `beq rs, rt, label` `branch if equal[I]`
  若 `rs == rt`，跳转至 `label`。
- `bne rs, rt, label` `branch if not equal[I]`
  若 `rs != rt`，跳转至 `label`。

- `bgez rs, label` branch if greater than or equal to zero[I]
  若 `rs >= 0`，跳转至 `label`。
- `bgezal rs, label` branch if greater than or equal to zero and link[I]
  若 `rs >= 0`，将返回地址存入 `$ra` 后跳转。
- `bgtz rs, label` branch if greater than zero[I]
  若 `rs > 0`，跳转。
- `blez rs, label` branch if less than or equal to zero[I]
  若 `rs <= 0`，跳转。
- `bltz rs, label` branch if less than zero[I]
  若 `rs < 0`，跳转。
- `bltzal rs, label` branch if less than zero and link[I]
  若 `rs < 0`，将返回地址存入 `$ra` 后跳转。

七、系统调用

- `syscall` [R]
  触发系统调用异常。

（贰）宏指令（伪指令）

▶ **abs rd, rs** —— 求绝对值```absolute value```

```
addu  rd, $zero, rs
bgez  rs, 1 #此处为行标号
sub   rd, $zero, rs
1:
```

先加到rd寄存器里，如果rd值大于等于0，无事，否则用0减自己

▶ **beqz rs, label** —— 等于零则跳转```branch if equal to zero```

```
beq   rs, $zero, label
```

▶ **bge rs, rt, label** —— 大于等于则跳转（有符号）

```
slt   $at, rs, rt
beq   $at, $zero, label
```

使用slt，大于等于即为不小于，即赋值0，再考虑与0的大小关系

▶ **bgeu rs, rt, label** —— 大于等于则跳转（无符号）`branch if greater or equal`

```
sltu  $at, rs, rt
beq   $at, $zero, label
```

▶ **bgt rs, rt, label** —— 大于则跳转（有符号）`branch if greater than`

```
slt   $at, rt, rs
bne   $at, $zero, label

判断rs是不是大于rt,即判断rt是不是小于rs，即判断`$at`是不是1,即判断`$at`是不是不等于0(因为只有判断是否为0的，故在判断是否为1时应当判断是否不等于0!)
```

▶ **bgtu rs, rt, label** —— 大于则跳转（无符号）

```
sltu  $at, rt, rs
bne   $at, $zero, label
```

▶ **ble rs, rt, label** —— 小于等于则跳转（有符号）`branch if less or equal`

```
slt   $at, rt, rs
beq   $at, $zero, label
```

▶ **bleu rs, rt, label** —— 小于等于则跳转（无符号）

```
sltu  $at, rt, rs
beq   $at, $zero, label
```

rs小于等于rt即为rt大于等于rs,即rt不小于rs,如果rt小于rs,则置0

▶ **blt rs, rt, label** —— 小于则跳转（有符号）`branch if less than`

```
slt   $at, rs, rt
bne   $at, $zero, label
```

▶ **bltu rs, rt, label** —— 小于则跳转（无符号）

```
sltu  $at, rs, rt
bne   $at, $zero, label
```

▶ **bnez rs, label** —— 不等于零则跳转`branch if not equal to zero`

```
bne   rs, $zero, label
```

▶ **b label** —— 无条件相对跳转

```
bgez  $zero, label
# 或
beq   $zero, $zero, label
```

9:24

clawX 含义解释

快速模式

1. beq / b 的跳转范围计算

beq 使用 **I 型指令格式**，其中用于计算跳转地址的立即数字段只有 **16 位**，并且是**有符号**的（可以向前或向后跳）。

其目标地址的计算公式为：

$$\text{新 PC} = (\text{PC} + 4) + (\text{立即数} \times 4)$$

- PC + 4：是**下一条指令**的地址（MIPS 的流水线特性）。
- 立即数：是一个 16 位有符号整数，范围是

-32768 到 +32767。

- `× 4`：因为指令是 4 字节对齐的，所以偏移量以“指令条数”为单位，而不是字节。

因此，`beq` 能跳转的**最大字节范围**是：

$$-32768 \times 4 \text{ 到 } +32767 \times 4 = \text{-128 KB 到 } +128$$

结论：`b label` 的跳转范围是当前指令所在位置的 ±128 KB。

---

2. 如果 `label` 超出 ±128 KB 范围怎么办？

如果跳转的目标 `label` 太远，汇编器就不能再把它翻译成 `beq $zero, $zero, label` 了，因为 `label` 的偏移量无法用一个 16 位有符号数表示。

这时，聪明的汇编器会采用另一种实现方式：
使用 `j`（无条件跳转）指令，或者 `jr` 指令。

例如，它可能会将 `b far_label` 展开为：

assembly 复制

```assembly
j far_label
```

`j` 指令是 **J 型格式**，它有一个 26 位的直接地址字段，可以实现 **256 MB** 范围内的跳转，范围要大得多。

---

3. 总结

▶ **div rd, rs, rt** —— 有符号除法（商）

```
bne   rt, $zero, ok
break $zero
ok:
div   rs, rt
mflo  rd
```

▶ **div rt, rs, imm(32)** —— 有符号除法（商）

```
li $at,imm  #实际上还要细分，li也有不同的展开，主要看imm的值
bne   $at, $zero, ok
break $zero
ok:
div   rs, $at
mflo  rt
```

▶ **divu rd, rs, rt** —— 无符号除法（商）

```
bne   rt, $zero, ok
break $zero
ok:
```

```
divu  rs, rt
mflo  rd
```

▶ **divu rt, rs, imm(32)** —— 无符号除法（商）

```
li $at,imm  #实际上还要细分，li也有不同的展开，主要看imm的值
bne   $at, $zero, ok
break $zero
ok:
divu   rs, $at
mflo  rt
```

▶ **la rd, label** —— 加载地址

```
lui   $at, %hi(label)
ori   rd, $at, %lo(label)
#符号 %hi() 和 %lo() 是 MIPS 汇编器提供的运算符，用来在汇编阶段把一个 32 位地址拆分成高 16 位和低 16 位。
```

▶ **li rd, value** （value ≥ 32768 或负数）

```
lui   $at, %hi(value)
ori   rd, $at, %lo(value)
```

一、为什么负数也能用 `lui` + `ori`？

因为 `lui` + `ori` 可以构造任意 32 位值，包括负数。
例如 `li $t0, -1`：

asm 复制 下载

```asm
lui  $t0, 0xFFFF
ori  $t0, $t0, 0xFFFF
```

有时候加载负数也可以使用:addiu

五、伪指令 `li` 加载负数时，为什么展开成 `addiu`？

asm 复制 下载

```asm
li $t0, -100
```

MARS 会展开为：

asm 复制 下载

```asm
addiu $t0, $zero, -100
```

- 因为 `addiu` 不检查溢出，且能处理负立即数
- 一条指令就能完成加载，比 `lui` + `ori` 更高效

▶ **li rd, value** （value < 32768）

```
ori   rd, $zero, value
```

PS:有时-3276value

▶ **move rd, rs** —— 寄存器间数据传送

```
addu  rd, $zero, rs
```

▶ **mul rd, rs, rt** —— 乘法（不检查溢出）

```
mult  rs, rt
mflo  rd
```

▶ **mul rt, rs, imm(32)** —— 乘法（不检查溢出）

```
li $at,imm
mult  rs, $at
mflo  rt
```

注意:

没有`mulu`的用法，因为不管有无符号，其低32位数都相同:

一、符号的定义

设两个 32 位二进制数 X 和 Y。

- 把 X 解释为有符号整数：$S(X)$
- 把 X 解释为无符号整数：$U(X)$

它们的关系是：

$$U(X) = S(X) + 2^{32} \cdot [X < 0]$$

即：如果 X 是负数（最高位为 1），就加上 2^{32}。

二、有符号乘积

$$P_s = S(X) \times S(Y)$$

三、无符号乘积

$$P_u = U(X) \times U(Y)$$

代入 $U(X) = S(X) + 2^{32} \cdot x_n$，其中 $x_n = 1$ 如果 X 为负，否则为 0。
同理 $U(Y) = S(Y) + 2^{32} \cdot y_n$。

展开：

$$P_u = [S(X) + 2^{32}x_n] \times [S(Y) + 2^{32}y_n]$$

$$= S(X)S(Y) + 2^{32}[x_nS(Y) + y_nS(X)] + 2^{64}x_ny_n$$

即：

$$P_u = P_s + 2^{32} \cdot K + 2^{64} \cdot x_ny_n$$

其中 $K = x_nS(Y) + y_nS(X)$ 是一个整数。

四、取低 32 位

注意：

- $2^{32} \cdot K$ 的低 32 位为 0
- $2^{64} \cdot x_ny_n$ 的低 32 位也为 0（因为 2^64 已经是 2^32 的倍数）

所以：

$$P_u \bmod 2^{32} = P_s \bmod 2^{32}$$

证毕。

五、4 位小例子验证（避免数字太大）

取 4 位二进制数：

一、为什么是 2^{32}，不是 2^{31}？

因为 32 位二进制数的范围是：

- 有符号：$-2^{31} \sim 2^{31} - 1$
- 无符号：$0 \sim 2^{32} - 1$

从有符号负数转到无符号时，加的是整个 32 位空间的跨度：2^{32}，不是 2^{31}。

那就是先加一个2^31进行非负，然后再加一个2^31（这是最高位视为数值位的操作）

▶ **mulo rd, rs, rt** —— 有符号乘法（检查溢出）

```
mult  rs, rt
mfhi  $at
mflo  rd
sra   rd, rd, 31
beq   $at, rd, ok
break $zero
ok:
mflo  rd
```

▶ **mulou rd, rs, rt** —— 无符号乘法（检查溢出）

```
multu rs, rt
mfhi  $at
beq   $at, $zero, ok
break $zero
ok:
mflo  rd
```

▶ **neg rd, rs** —— 求补（有符号，检查溢出）

```
sub   rd, $zero, rs
```

▶ **negu rd, rs** —— 求补（无符号）

```
subu  rd, $zero, rs
```

▶ **nop** —— 空操作

```
or    $zero, $zero, $zero
```

▶ **not rd, rs** —— 按位取反

```
nor   rd, rs, $zero
```

▶ **rem rd, rs, rt** —— 有符号除法（余数）`remain`

```
bne   rt, $zero, ok
break $zero
ok:
div   rs, rt
mfhi  rd
```

注意老式写法：

```
bne rt,$0,8  # 如果 rt != 0，跳过 8 字节即 2 条指令（即直接跳到 mfhi 行）
break $0
div rs,rt
mfhi rd
```

▶ **rem rt, rs, imm(32)** —— 无符号除法（余数）

```
li $at,imm
bne   $at, $zero, ok
break $zero
ok:
```

```
div  rs, $at
mfhi  rt
```

▶ **remu rd, rs, rt** —— 无符号除法（余数）

```
bne   rt, $zero, ok
break $zero
ok:
divu  rs, rt
mfhi  rd
```

▶ **remu rt, rs, imm(32)** —— 无符号除法（余数）

```
li $at,imm
bne   $at, $zero, ok
break $zero
ok:
divu  rs, $at
mfhi  rt
```

▶ **rol rd, rs, rt** —— 循环左移（变量移位）`rotate left`

```
subu  $at, $zero, rt # $at = -rt (补码), 其低5位等效于 32 - rt
#主要是因为我们要32-rt,但是这样的话，我们没有在rs上放立即数的方式
不然我们就要用一个寄存器存放固定值32了
srlv  $at, rs, $at
sllv  rd, rs, rt
or    rd, rd, $at
```

2. 巧用补码

`subu $at, $zero, rt` → `$at = -rt`

在 MIPS 中，移位指令（`sllv` / `srlv`）只取移位量寄存器的**低 5 位**（0–31）。

`-rt` 的低 5 位 = `(32 - (rt mod 32)) mod 32`

如果 `rt` 在 0–31 之间，`-rt` 的低 5 位 = `(32 - rt) mod 32`。
当 `rt` 不是 0 时，`(32 - rt) mod 32` 就是 `32 - rt`。

所以 `-rt` 的低 5 位 == `32 - rt`（模 32 意义下）。

▶ **rol rd, rs, sa** —— 循环左移（固定移位）

```
srl   $at, rs, 32-sa
sll   rd, rs, sa
or    rd, rd, $at
```

▶ **ror rd, rs, rt** —— 循环右移（变量移位）`rotate right`

```
subu  $at, $zero, rt
sllv  $at, rs, $at
srlv  rd, rs, rt
or    rd, rd, $at
```

▶ **ror rd, rs, sa** —— 循环右移（固定移位）

```
sll   $at, rs, 32-sa
srl   rd, rs, sa
or    rd, rd, $at
```

▶ **seq rd, rs, rt** —— 相等则置 1`set if equal`

```
beq   rt, rs, yes
ori   rd, $zero, 0
beq   $zero, $zero, skip
yes:
ori   rd, $zero, 1
skip:
```

▶ **sge rd, rs, rt** —— 大于等于则置 1（有符号）`set if greater or equal`

```
bne   rt, rs, yes
ori   rd, $zero, 1
beq   $zero, $zero, skip
yes:
slt   rd, rt, rs
skip:
```

▶ **sgeu rd, rs, rt** —— 大于等于则置 1（无符号）

```
bne   rt, rs, yes
ori   rd, $zero, 1
beq   $zero, $zero, skip
yes:
sltu  rd, rt, rs
skip:
```

▶ **sgt rd, rs, rt** —— 大于则置 1（有符号）`set if greater than`

```
slt   rd, rt, rs
```

▶ **sgtu rd, rs, rt** —— 大于则置 1（无符号）

```
sltu  rd, rt, rs
```

▶ **sle rd, rs, rt** —— 小于等于则置 1（有符号）`set if less or equal`

```
bne   rt, rs, yes
ori   rd, $zero, 1
beq   $zero, $zero, skip
yes:
slt   rd, rs, rt
skip:
```

▶ **sleu rd, rs, rt** —— 小于等于则置 1（无符号）

```
bne   rt, rs, yes
ori   rd, $zero, 1
beq   $zero, $zero, skip
yes:
sltu  rd, rs, rt
skip:
```

▶ **sne rd, rs, rt** —— 不等则置 1`set if not equal`

```
beq   rt, rs, yes
ori   rd, $zero, 1
beq   $zero, $zero, skip
yes:
ori   rd, $zero, 0
skip:
```

MIPS 通用寄存器表（32 个）

编号	助记符	用途说明	英文全称 / 翻译
0	`$zero`	恒为 0，写入无效	**zero** constant
1	`$at`	汇编器临时使用（展开伪指令）	**a**ssembler **t**emporary
2-3	`$v0` ~ `$v1`	函数返回值	**v**alue returned
4-7	`$a0` ~ `$a3`	函数实参（前 4 个）	**a**rguments
8-15	`$t0` ~ `$t7`	临时寄存器，调用者保存	**t**emporary
16-23	`$s0` ~ `$s7`	保存寄存器，被调用者保存	**s**aved
24-25	`$t8` ~ `$t9`	临时寄存器（同 t0~t7）	**t**emporary
26-27	`$k0` ~ `$k1`	保留给 OS 内核，异常处理用	**k**ernel reserved

编号	助记符	用途说明	英文全称 / 翻译
28	`$gp`	全局数据区指针	**g**lobal **p**ointer
29	`$sp`	栈顶指针	**s**tack **p**ointer
30	`$fp` / `$s8`	帧指针（或作为第 9 个保存寄存器）	**f**rame **p**ointer / **s**aved
31	`$ra`	函数返回地址	**r**eturn **a**ddress

特殊寄存器（非通用寄存器）

名称	用途说明	翻译
PC	程序计数器，存放下一条指令地址	**P**rogram **C**ounter
HI	乘法结果高 32 位 / 除法余数	**HI**gh word
LO	乘法结果低 32 位 / 除法商	**LO**w word
IR	指令寄存器，存放当前执行的机器码	**I**nstruction **R**egister

MIPS 架构与指令集问答

第一部分：概述与 MIPS 架构

▶ Q1：和高级语言相比，低级语言有何优缺点？

- **优点**：
  - 执行效率高，能够直接访问系统接口。
  - 程序体积小，适合嵌入式或对资源敏感的环境。
  - 能够实现高级语言无法做到的底层操作（如上下文切换、中断处理）。
- **缺点**：
  - 可读性差，开发效率低，维护困难。
  - 与硬件平台强相关，移植性差。
  - 容易出错且调试困难。

▶ Q2：为何需要学习汇编语言？

- 对于计算机技术的初学者，编写汇编语言程序可以深入了解计算机的程序执行过程，理解计算机底层工作原理，有助于对高级语言程序机制的理解。
- 直观感受CPU的结构和指令执行，有助于后期硬件相关课程的学习。
- 帮助调试和优化高级语言程序（如分析 C 语言反汇编）。
- 编写操作系统内核、驱动程序、嵌入式系统的必需技能。
- 应对安全领域中的逆向工程和漏洞分析需求。

▶ Q3：汇编源程序和汇编程序分别是什么？

- **汇编源程序**：使用汇编语言编写的程序。
- **汇编程序**：将汇编源程序翻译成机器码的工具软件（如 MARS, SPIM, GNU `as`），也叫**汇编器/解释器**。

▶ **Q4：根据机器指令体系，CPU 分为哪两大类？典型代表有哪些？**

- **CISC**（complex instruction set computer复杂指令集计算机）：指令数量多、长度可变、单条指令功能强。
  代表：x86（Intel、AMD）。
- **RISC**（reduced instruction set computer精简指令集计算机）：指令数量少、长度固定、大部分指令单周期执行。
  代表：MIPS、ARM、RISC-V。

▶ **架构 Q1：学习 MIPS 架构需要了解哪四个主要方面的内容？**

1. **各类寄存器**：通用寄存器、专用寄存器(HI/LO)、特殊寄存器（PC/IR）。
2. **指令集与指令格式**：R/I/J 三种格式的字段划分与功能。
3. **内存寻址模式**：立即数、寄存器、基址偏移、伪直接、相对寻址。
4. **数据类型与存储格式**：字节、半字、字，以及大/小端对齐。

▶ **架构 Q2：数据类型和高级语言中的数据类型有何不同？**

- 高级语言数据类型（如 `int`, `char`, `float`）带有**语义和检查**（如不能把 `float` 直接当指针用）。
- 汇编语言的数据类型只是**二进制位串的长度约定**（如 8 位、16 位、32 位），硬件不检查类型是否匹配，只按指令操作。

▶ **架构 Q3：MIPS 架构中通用寄存器有多少个？一个寄存器多少位？**

- **32 个**通用寄存器（`$0` ~ `$31`）。
- 每个寄存器 **32 位**（4 字节）。

▶ **架构 Q4：用于传递函数输入实参的寄存器是哪些？**

`$a0` ~ `$a3`（编号 4~7）。超过 4 个参数时，其余参数通过堆栈传递。

▶ **架构 Q5：用于存放函数返回值的寄存器是哪些？**

`$v0` ~ `$v1`（编号 2~3）。通常 32 位返回值仅用 `$v0`，64 位返回值使用 `$v0` 和 `$v1` 共同存放。

▶ **架构 Q6：Main 调用 A，Main 中 t0 存有重要值，A 会修改 t0，如何保证 t0 不被改变？**

`$t0` ~ `$t9` 是**临时寄存器**，按照 MIPS 调用约定，**被调用函数（A）不需要保存它们。**
因此 **Main 必须在调用 A 之前自己把 `$t0` 保存到栈中**，A 返回后再从栈中恢复。

```
addiu $sp, $sp, -4
sw    $t0, 0($sp)      # 保存 t0
jal   A
lw    $t0, 0($sp)      # 恢复 t0
addiu $sp, $sp, 4
```

对比：如果用的是 `$s0`

反过来，如果 Main 把重要值存在 `$s0` 中：

```asm
Main:
    # 直接调用即可，无需保存
    jal   A
    # s0 的值一定没变
```

因为按照约定，`A` 如果想用 `$s0`，**必须在 A 的开头保存、结尾恢复**：

```asm
A:
    addiu $sp, $sp, -4
    sw    $s0, 0($sp)     # A 主动保存 s0

    # ... A 随便用 s0 ...

    lw    $s0, 0($sp)     # A 恢复 s0
    addiu $sp, $sp, 4
    jr    $ra
```

一句话总结

> **`$t0` 是临时工，Main 要用就得自己存；`$s0` 是正式工，A 用了必须原样归还。**

把 `$t0` 的值复制到另一个寄存器，确实能"保住"这个值——前提是**目标寄存器本身不会被 `A` 破坏。**

两种可行的寄存器选择

方案一：挪到 `$s0` ~ `$s7`（推荐）

```asm
Main:
    move  $s0, $t0    # 把重要值从 t0 搬到 s0
    jal   A           # A 可以随便破坏 t0
    move  $t0, $s0    # 从 s0 搬回来继续用
```

- **为什么安全**：按照调用约定，`A` 如果用 `$s0`，必须自己保存恢复。Main 不用操心。
- **代价**：占用了一个 `$s` 寄存器。

方案二：挪到另一个 `$t` 寄存器（不保险）

asm 复制 下载

```asm
Main:
    move  $t5, $t0    # 搬到 t5
    jal   A
    move  $t0, $t5    # 搬回来
```

- **为什么可能不安全**：`A` 破坏 `$t5` 也是完全合法的。你只是赌 `A` 没用 `$t5`。
- **结论**：只有在你**确信** `A` 不会碰 `$t5` 时才能用（比如 `A` 是你自己写的、代码很短）。

保存到栈 vs 挪到其他寄存器

可以理解为函数传形参、引用等

▶ **架构 Q7：若 Main 使用 s0 存放重要值，该如何做？**

`$s0` ~ `$s7` 是**保存寄存器**，调用约定规定**被调用函数（A）必须保证这些寄存器在返回时与调用前一致。**
因此 **Main 无需额外保存**，A 若需要使用 `$s0`，会在自己的代码开头保存并在返回前恢复。

用 $s0（被调用者保存）可以让函数“黑盒化”：函数内部随便用 $s0，返回后外面的值不变，函数可以放心地被重复调用。

Main 侧无需任何操作，直接调用即可。

▶ **架构 Q8：保留给 OS 使用的寄存器是哪些？保留给汇编程序使用的寄存器是？**

- **OS 保留**：`$k0`, `$k1`（编号 26~27），用于异常处理。
- **汇编程序保留**：`$at`（编号 1），汇编器在展开伪指令时临时使用。
- 在笔记中，宏指令展开大量使用 `$at`。

▶ **架构 Q9：用于存放函数返回地址的寄存器是哪个？存放栈顶地址的寄存器是哪个？**

- 返回地址：`$ra`（编号 31）。
- 栈顶地址：`$sp`（编号 29）。

▶ **架构 Q10：HI 寄存器用于存放什么数据？LO 寄存器用于存放什么数据？**

- **HI**：存放乘法结果的高 32 位，或除法的余数。
- **LO**：存放乘法结果的低 32 位，或除法的商。

▶ **架构 Q11：简述汇编源程序执行过程？**

1. **编辑**：编写 `.asm` 源文件。
2. **汇编**：汇编器将助记符翻译成机器码，生成目标文件（`.o` 或 `.obj`）。
3. **链接**：链接器将多个目标文件和库合并，解析地址，生成可执行文件。
4. **加载**：操作系统将可执行文件载入内存。
5. **执行**：CPU 从 `_start` 或 `main` 入口开始逐条取指、译码、执行。

▶ 架构 Q12：程序计数器寄存器 PC 存放什么值？为何一次递增 4？何时递增？

- **存放**：当前正在执行指令的地址（MIPS 中通常指向下一条要取的指令）。
- **递增 4**：因为 MIPS 指令固定 32 位（4 字节），地址按字节编址，所以每条指令地址间隔为 4。
- **递增时机**：在取指阶段（IF）完成后，PC 自动加 4 指向下一条顺序指令。

▶ 架构 Q13：画一个 64KB 的存储器。

64KB = 2^16 字节，地址范围从 `0x0000` 到 `0xFFFF`。
通常分为：

- **Text 段**：存放代码（低地址端）。
- **Data 段**：存放已初始化全局变量。
- **BSS 段**：存放未初始化全局变量。
- **Heap**：动态分配区（向上增长）。
- **Stack**：栈区（向下增长，从高地址开始）。

简易图示（按字节编址）：

```
0x0000 ┌──────────┐
       │   Text   │ 代码段
0x???? ├──────────┤
       │   Data   │ 已初始化数据
0x???? ├──────────┤
       │   BSS    │ 未初始化数据
0x???? ├──────────┤
       │   Heap   │ → 向高地址增长
       │   ...    │
       │   Stack  │ ← 向低地址增长
0xFFFF └──────────┘
```

▶ 架构 Q14：现有存储器，对齐要求下，取出地址为 0003 的一个字节、0002 的一个半字、0001 的一个字。

- **取字节 `0x0003`**：对齐无要求，可直接读取该地址 1 字节。
- **取半字 `0x0002`**：半字对齐要求地址为 2 的倍数，`0x0002` 符合，可读取 `0x0002` 和 `0x0003` 两个字节组成半字。
- **取字 `0x0001`**：字对齐要求地址为 4 的倍数，`0x0001` **不符合**，将引发**地址错误异常**。

▶ 架构 Q15：内存分为几个部分？

典型 MIPS 进程内存布局（从低地址到高地址）：

1. **Text**（代码段）
2. **Data**（已初始化数据段）

3. **BSS**（未初始化数据段）
4. **Heap**（堆，动态分配，向上增长）
5. **Stack**（栈，局部变量，向下增长）
6. **Kernel Space**（内核空间，用户态不可访问）

▶ **架构 Q16：指令寄存器 IR 存放什么？MIPS 指令有几种格式？**

- **IR**：存放当前正在**译码和执行**的机器指令（32 位二进制码）。
- **格式种类**：**3 种** —— R 格式、I 格式、J 格式。

▶ **架构 Q17：R 指令有几段？举一个指令例子。**

R 格式共 6 个字段：

op (6)	rs (5)	rt (5)	rd (5)	sa (5)	funct (6)

例子：`add $t0, $t1, $t2`
op=`000000`, funct=`100000`。

▶ **架构 Q18：I 指令有几段？举一个指令例子。**

I 格式共 4 个字段：

op (6)	rs (5)	rt (5)	immediate (16)

例子：`lw $t0, 4($sp)`
op=`100011`, rs=`$sp`, rt=`$t0`, imm=`0004`。

▶ **架构 Q19：J 指令有几段？举一个指令例子。**

J 格式共 2 个字段：

op (6)	address (26)

例子：`j label`
op=`000010`, address 为目标地址的高 26 位（实际地址需左移 2 位后与 PC 高 4 位拼接）。

▶ **架构 Q20：结合指令动画理解三种格式指令并描述指令执行过程。**

以 R 型 `add $t0, $t1, $t2` 为例：

1. **取指IF$(Instruction\ Fetch)$**：PC 所指指令读入 IR，PC+4 -> PC。
2. **译码RD$(Register\ Decode\ /\ Read)$**：解析IR中指令，识别为 R 型，从 `rs`、`rt` 读取 `$t1`、`$t2` 的值。
3. **执行ALU$(Arithmetic\ Logic\ Unit\ Execute)$**：ALU 将两数相加，得到结果。
4. **访存MEM$(Memory\ Access)$**：R 型无访存，结果直通下一阶段。
5. **写回WB$(Write\ Back)$**：结果写入 `rd` 指定的 `$t0`。

I 型和 J 型在地址计算和目标写入上有所区别，但基本五阶段流水线结构相同。

第二部分：寻址方式与指令

▶ **寻址方式 Q1：操作数寻址方式有哪几种？分别是什么含义？**

MIPS 支持 5 种操作数寻址方式： **针对操作数的寻址**

1. **立即数寻址$(immediate\ addressing)$**：源操作数之一为立即数，目的操作数为寄存器。(例 `addi $t1,$t2,5`)
2. **寄存器寻址$(register\ addressing)$**：操作数在寄存器中。(例`add $t0,$t1,$t3`) **存储单元寻址**
3. **基址寻址**：操作数在内存中，地址 = 寄存器 + 16 位偏移量（`lw/sw`）。

> 基地址(存放在某个【通用】寄存器中)+位移量(在指令中以16bits补码数存放)$(base\ addressing+displacement)$

基址寻址举例

- lw $t2, 32($t0)
  - ✓ 假设寄存器$t0中的值为100，则源操作数所在存储单元起始地址为100+32=132（0x00000084）

字段值（I指令格式）			
opcode	rs	rt	imm
35	8	10	32
6bits	5bits	5bit	16bits

机器码（**0x8D0A0020**）			
opcode	Rs	rt	imm
100011	01000	01010	0000000000100000
6bits	5bits	5bit	16bits

基址寻址举例

- sw $s1, -32($t1)
  - ✓ 假设寄存器$t1中的值为100，则目的操作数所在存储单元起始地址为100-32=68（0x00000044）

字段值（I指令格式）			
opcode	rs	rt	imm
43	9	17	-32
6bits	5bits	5bit	16bits

机器码（**0xAD31FFE0**）			
opcode	rs	rt	imm
101011	01001	10001	1111111111100000
6bits	5bits	5bit	16bits

六、总结

> **基址寻址的立即数（偏移量）一定是符号扩展，因为地址计算需要支持负偏移。**
> 而运算指令里的立即数扩展方式，取决于该指令是算术/比较（符号扩展）还是逻辑（零扩展）。

你之前的理解“立即数寻址和基址寻址都是符号扩展”对**基址寻址**完全正确，但对**立即数寻址**要分情况。

针对目标地址的寻址 *分支转移:*

> 实现高级语言的分支、循环以及函数调用与返回等语言成分必不可少的操作。
> 实质时改变了程序顺序执行指令的行为（PC增量定值的行为），通过修改PC值实现分支转移功能。

4. **伪直接寻址$(Pseudodirect\ addressing)$**：用于 `j`（无条件转移指令）/`jal`（无条件转移指令并链接），26 位地址左移 2 位与 PC 高 4 位拼接。

5. **寄存器间接寻址**

寄存器间接寻址

- 某个通用寄存器内存放的是目标地址
- 转移指令执行时取出寄存器内存放的目标地址放入PC中，即实现跳转。
- 使用R指令格式
- MIPS架构设置有寄存器跳转（jump register）指令jr
  - ✓ 无条件寄存器间接跳转： jr rs，例如jr $ra
    - 指令行为； PC = R[rs]
  - ✓ 无条件跳转并寄存器链接（函数（子程序）调用）： jalr rd, rs，例如jalr $ra,$a0
    - 指令行为； R[rd] = PC; PC = R[rs]

6. **(PC)相对寻址方式**：用于分支指令，目标地址 = PC + 4 + (imm << 2)。`

- MIPS指令中条件转移指令使用的目标地址寻址方式
  - ✓ 例如：beq rs, rt, Label # rs, rt寄存器内存放的的数值相等，则分支转移到标号Label处继续执行指令，否则顺序执行下一条指令。
  - ✓ 使用的是I指令格式
  - ✓ 指令执行的操作：
    - If (R[rs] == R[rt])then PC =PC + Imm<< 2
    - 假设beq指令所在的内存地址为x，标号Label代表的内存地址为y，则Imm=（y-x-4）>> 2

实际上是I指令格式，imm在静态编译时，通过公式imm=(y-x-4)>>2（当前地址与label中间隔imm条指令）算得。然后在动态运行时在IR中填入这个imm

寻址方式	操作数位置	寻址过程	指令格式	MIPS 典型指令	通俗类比	为何叫这个名字
立即数寻址	指令内部	直接从指令码中提取常数	I 型	`addi $t0, $t1, 5`	口袋里直接摸出 5 块钱	数据**立即**可得，就在指令里
寄存器寻址	寄存器	直接读寄存器文件	R 型	`add $t0, $t1, $t2`	从抽屉里拿东西	数据在**寄存器**里，选号即用
基址寻址 (基址+偏移)	内存	地址 = 寄存器值 + 偏移量	I 型	`lw $t0, 100($t1)`	以书架第一层为基准，往右数 10 本书	寄存器提供**基地址**，偏移量定位具体位置
寄存器间接寻址	内存	地址 = 寄存器值	R 型 (跳转类)	`jr $ra` `jalr $t0`	纸条上写着门牌号，按号去找	不直接给数据，给的是**存放数据的地址**（间接）
伪直接寻址	指令附近 (256MB 内)	地址 = PC[31:28] || imm<<2	J 型	`j label` `jal label`	说"大厦 12 楼"，默认还在本栋楼	看起来像直接给 26 位地址，实则要靠 PC 高 4 位拼凑（**伪**）
相对寻址	指令附近	地址 = PC + 4 + imm<<2	I 型	`beq $t0, $t1, label`	"往前走 3 步"而不是"去 403 房间"	目标是**相对于当前 PC** 的位移量

▶ 寻址方式 Q2：采用立即数进行操作数寻址的指令是什么格式？采用寄存器进行操作数寻址的指令是什么格式？

- **立即数寻址**：I 格式（如 `addi`, `ori`, `lui`）。
- **寄存器寻址**：R 格式（如 `add`, `sub`, `and`）和部分 I 格式（如 `beq` 的源操作数）。

▶ 寻址方式 Q3：采用存储单元进行操作数寻址的指令是什么格式？什么写法？可以直接访问内存的指令有哪些？

- **格式**：I 格式。
- **写法**：`offset(rs)`，如 `lw $t0, 8($sp)`。
- **直接访存指令**：`lb`, `lbu`, `lh`, `lhu`, `lw`, `sb`, `sh`, `sw`。

▶ 寻址方式 Q4：为何需要目标地址寻址？

用于改变程序控制流（跳转和分支）。MIPS 中目标地址寻址分为：

- **伪直接寻址**（`j`, `jal`）：快速跳转到 256MB 范围内的绝对地址。
- **寄存器间接寻址**
- **相对寻址**（`beq`, `bne` 等）：相对于当前 PC 的偏移跳转，适合短距离条件分支。

▶ **寻址方式 Q5：寄存器间接寻址为何称为间接？采用寄存器间接寻址的指令是什么格式？**

- **间接含义**：指令中给出的寄存器**不包含操作数本身，而包含操作数的地址。**
- **格式**：R 格式。
  MIPS 中通过 `jr $ra` 或 `jalr $t0` 实现寄存器间接跳转，即 PC ← `rs`。

▶ **寻址方式 Q6：伪直接寻址为何称为伪？采用伪直接寻址的指令是什么格式？**

- **称为"伪"**：因为指令中的 26 位立即数并非完整的目标地址，必须**与 PC 高 4 位拼接**才能得到真正的 32 位地址，并非"直接"给出全地址。
- **格式**：J 格式（`j`, `jal`）。

▶ **寻址方式 Q7：相对寻址为何称为相对？采用相对寻址的指令是什么格式？如何计算 Imm？**

- **称为"相对"**：因为目标地址是**相对于当前 PC 的值**，而不是绝对地址。
- **格式**：I 格式（所有条件分支指令）。
- **Imm 计算**：
  机器码中的 `imm` = (目标地址 - (当前 PC + 4)) >> 2。
  汇编器会自动计算，程序员只需写 `label`。

▶ **数据传送类指令 Q8：从内存取数据的指令有哪些？它们是什么种类的指令？**

- 指令：`lb`, `lbu`, `lh`, `lhu`, `lw`
- 种类：I 格式。

▶ **数据传送类指令 Q9：带 u 和不带 u 指令的区别是什么？为何没有 lwu？**

- **区别**：加载的数据宽度小于 32 位时，**不带 u** 进行**符号扩展**至 32 位；**带 u** 进行**0扩展。**
- **没有 `lwu`**：因为 `lw` 本身就是加载 32 位字，**已经填满目标寄存器**，无需扩展。

▶ **数据传送类指令 Q10：把寄存器中数据存放到 Mem 的指令有哪些？它们是什么种类的指令？**

- 指令：`sb`, `sh`, `sw`
- 种类：I 格式。

▶ **数据传送类指令 Q11：把 HI 寄存器值传送到通用寄存器的指令是什么？该指令是什么格式？把通用寄存器值传送到 LO 寄存器的指令是什么？**

- **HI → GPR**：`mfhi rd`，R 格式。
- **GPR → LO**：`mtlo rs`，R 格式。

▶ **数据传送类指令 Q12：什么是宏指令？**

宏指令（伪指令）是汇编器提供的便利助记符，**在硬件中不存在对应的机器码。**汇编时会被展开为一条或多条真实的机器指令。例如 `li $t0, 100` → `addiu $t0, $zero, 100`。

▶ **数据传送类指令 Q13：如何把 32 位 Imm 存放到 Mem 中？**

无法用单条指令直接将 32 位立即数写入内存，需要分两步：

1. 将 32 位立即数加载到寄存器（`lui` + `ori`）。
2. 用 `sw` 将寄存器值存入内存。

```
lui $at, 0x1234
ori $at, $at, 0x5678   # $at = 0x12345678
sw  $at, 0($t0)        # 存入 t0 指向的内存
```

▶ **数据传送类指令 Q14：如何把 16 位立即数 Imm 存放到 Mem 中？**

16 位立即数可用 `ori` 或 `addiu` 直接加载到寄存器，再 `sw` 存出：

```
ori $at, $zero, 0x1234
sw  $at, 0($t0)
```

▶ **数据传送类指令 Q15：写出宏指令 li $10, 0xfffffff 的实现代码。**

`0xfffffff` 实际是 `0x0fffffff`（28 位有效），需要 `lui` + `ori`：

```
lui $t0, 0x0fff    # 高 16 位: 0x0fff
ori $t0, $t0, 0xffff   # 低 16 位: 0xffff
```

最终 `$t0` = `0x0fffffff`。

▶ **数据传送类指令 Q16：通用寄存器之间传送数据的宏指令是什么？其如何实现？**

- 宏指令：`move rd, rs`
- 实现：`addu rd, $zero, rs`
  （将 `rs` 加 0 的结果放入 `rd`）

▶ **算数运算类指令 Q17：加法指令包括哪些？分别是什么类型的指令？带 u 和不带 u 有何区别？**

- **R 型**：`add`, `addu`
- **I 型**：`addi`, `addiu`
- **区别**：
  - 带 `u`：不检测补码溢出，结果直接截断 32 位。
  - 不带 `u`：可能引发溢出异常（取决于具体实现）。

▶ **算数运算类指令 Q18：减法指令包括哪些？分别是什么类型的指令？**

- `sub`：R 型，检测溢出。
- `subu`：R 型，不检测溢出。
- **没有 I 型减法。**

▶ **算数运算类指令 Q19：减法指令为何不像加法指令那样有带 XXi 型指令？如何实现减去一个立即数？**

- **原因**：减法可以通过**加上一个负数立即数**实现，没必要单独设计 `subi` 指令，节省了指令编码空间。
- **实现**：`addiu rt, rs, -imm`
  因为立即数在 `addiu` 中会【符号扩展】，负数可以正确参与运算。

▶ **算数运算类指令 Q20：与减法指令相关的宏指令有哪些？**

- `neg rd, rs`（求补）：展开为 `sub rd, $zero, rs`。
- `negu rd, rs`（无符号求补）：展开为 `subu rd, $zero, rs`。

▶ 算数运算类指令 Q21：求补宏指令能否对 -2^31 求补？

不能。
`-2^31` 的补码是 `0x80000000`，其相反数 `2^31` 在 32 位有符号整数中**溢出**（最大正值是 `2^31-1`）。因此对 `0x80000000` 求补结果仍是自身，且带溢出检测的 `neg` 会触发异常。

▶ 算数运算类指令 Q22：乘/除法指令包括哪些？是什么类型的指令？

- 乘法：`mult` (有符号), `multu` (无符号) —— R 格式。
- 除法：`div` (有符号), `divu` (无符号) —— R 格式。

▶ 算数运算类指令 Q23：与乘法指令相关的宏指令有哪些？

- `mul rd, rs, rt`：`mult` + `mflo`。
- `mulo rd, rs, rt`：带溢出检查的乘法，若 `HI` ≠ `rd` 的【符号扩展】则溢出。
- `mulou rd, rs, rt`：无符号乘法溢出检查，若 `HI` ≠ 0 则溢出。

▶ 算数运算类指令 Q24：与除法指令相关的宏指令有哪些？

- `div rd, rs, rt`：`div rs, rt` + `mflo rd`。
- `divu rd, rs, rt`：`divu rs, rt` + `mflo rd`。
- `rem rd, rs, rt`：`div` + `mfhi`。
- `remu rd, rs, rt`：`divu` + `mfhi`。

▶ 逻辑运算类指令 Q25：与运算指令包括哪些？是什么类型的指令？

- `and`：R 型。
- `andi`：I 型（立即数 0扩展）。

▶ 逻辑运算类指令 Q26：或运算指令包括哪些？是什么类型的指令？

- `or`：R 型。
- `ori`：I 型（立即数 0扩展）。

▶ 逻辑运算类指令 Q27：异或运算指令包括哪些？是什么类型的指令？

- `xor`：R 型。
- `xori`：I 型（立即数 0扩展）。

▶ 逻辑运算类指令 Q28：或非指令含义是什么？非运算如何实现？

- **或非 `nor`**：对 `rs` 和 `rt` 按位或后取反，R 格式。
- **非运算**：MIPS 无原生 `not`，使用宏指令 `not rd, rs` 展开为 `nor rd, rs, $zero`。

▶ 移位指令 Q29：逻辑左移指令包括哪些？是什么类型的指令？

- `sll`：移位量固定（`sa` 字段），R 格式。
- `sllv`：移位量可变（取自 `rs` 低 5 位），R 格式。

▶ 移位指令 Q30：逻辑右移指令包括哪些？是什么类型的指令？

- `srl`：固定移位量，R 格式。
- `srlv`：可变移位量，R 格式。

▶ 移位指令 Q31：算术右移指令包括哪些？是什么类型的指令？

- `sra`：固定移位量，R 格式。
- `srav`：可变移位量，R 格式。

▶ 移位指令 Q32：为何没有算术左移指令？如何实现算术左移？

- **原因**：**逻辑左移和算术左移效果完全一致**（最低位补 0，符号位自然移出），因此无需单独设计。
- **实现**：直接用 `sll` 或 `sllv` 即可。

▶ 移位指令 Q33：循环左移宏指令如何实现？

`rol rd, rs, rt`（`rt` 指定移位数）：

```
subu $at, $zero, rt   # $at = -rt (补码), 其低5位等效于 32 - rt
srlv $at, rs, $at     # 取出将被移出的高位部分并右移对齐
sllv rd, rs, rt       # 左移剩余部分
or   rd, rd, $at      # 合并
```

> MIPS 进行可变移位（如 srlv）时，只读取 rs 寄存器的低 5 位（即 rt mod 32）。
> 假设我们要计算 32 - rt:
> 在 32 位补码运算中，-rt 的二进制低 5 位，恰好等于 (32 - rt) mod 32 的低 5 位。(只截取低5位，且无符号)

▶ 移位指令 Q34：循环右移宏指令如何实现？

`ror rd, rs, rt`:

```
subu $at, $zero, rt   # $at = 32 - rt
sllv $at, rs, $at     # 取出将被移出的低位部分并左移对齐
srlv rd, rs, rt       # 右移剩余部分
or   rd, rd, $at
```

▶ 条件设置指令 Q35：条件设置指令包括哪些？是什么类型的指令？

- **R 型**：`slt`, `sltu`
- **I 型**：`slti`, `sltiu`

▶ 条件设置指令 Q36：条件设置宏指令有哪些？

- `seq`, `sne`, `sgt`, `sgtu`, `sge`, `sgeu`, `sle`, `sleu`
- 它们都基于 `slt`/`sltu` 及分支指令组合实现。

▶ 跳转指令 Q37：无条件转移指令包括哪些？是什么类型的指令？

- `j`, `jal`：J 格式。

- `jr`, `jalr`：R 格式。

▶ 跳转指令 Q38：条件转移指令包括哪些？是什么类型的指令？

均为 I 格式：

- 与 0 比较：`bgez`, `bgezal`, `bgtz`, `blez`, `bltz`, `bltzal`
- 两寄存器比较：`beq`, `bne`

▶ 跳转指令 Q39：无条件转移宏指令如何实现？与 j 指令有何不同？

- 宏指令 `b label`：展开为 `bgez $zero, label` 或 `beq $zero, $zero, label`。
- **不同**：`b label` 是**相对寻址**，跳转范围有限（±128KB）；`j label` 是**伪直接寻址**，范围大（256MB）。两者寻址方式不同。

▶ 跳转指令 Q40：条件转移宏指令包括哪些？如何实现的？

- 宏指令：`beqz`, `bnez`, `bge`, `bgeu`, `bgt`, `bgtu`, `ble`, `bleu`, `blt`, `bltu`
- 实现方式：结合 `slt`/`sltu` 与 `beq`/`bne`，例如 `blt rs, rt, label`：

```
slt $at, rs, rt
bne $at, $zero, label
```

▶ 系统调用指令 Q41：系统调用指令是哪个？如何使用该指令？

- 指令：`syscall`（R 格式，funct=`001100`）。
- 使用步骤：
  1. 将系统调用号存入 `$v0`。
  2. 将参数存入 `$a0` ~ `$a3`。
  3. 执行 `syscall`。
  4. 返回值通常从 `$v0` 读取。

例如打印整数：

```
li $v0, 1       # 调用号 1: print_int
move $a0, $t0   # 要打印的值
syscall
```

MARS 常用 SYSCALL 服务表

服务名称	服务代码 ($v0)	需要的参数	返回值	功能说明
print_int	1	`$a0` = 要打印的整数	无	以十进制形式打印一个整数到控制台 [citation:5]

服务名称	服务代码 ($v0)	需要的参数	返回值	功能说明
print_float	2	`$f12` = 要打印的单精度浮点数	无	打印一个单精度浮点数 [citation:5]
print_double	3	`$f12` = 要打印的双精度浮点数	无	打印一个双精度浮点数 [citation:5]
print_string	4	`$a0` = 要打印的字符串地址 (以 null 结尾)	无	打印一个字符串到控制台 [citation:2]
read_int	5	无	`$v0` = 读取的整数	从控制台读取一行输入，并将其解析为整数返回 [citation:2]
read_float	6	无	`$f0` = 读取的浮点数	读取一个单精度浮点数 [citation:5]
read_double	7	无	`$f0` = 读取的双精度浮点数	读取一个双精度浮点数 [citation:5]
read_string	8	`$a0` = 存储字符串的缓冲区地址 `$a1` = 缓冲区长度（字节数）	无 (数据写入缓冲区)	从控制台读取一行字符串到指定缓冲区 [citation:2]
sbrk	9	`$a0` = 需要分配的字节数	`$v0` = 分配的内存块首地址	在堆上分配一块内存 [citation:5]
exit	10	无	无	终止程序执行 [citation:2]
print_char	11	`$a0` = 要打印的字符 (ASCII码)	无	打印一个字符到控制台 [citation:5]
read_char	12	无	`$v0` = 读取的字符 (ASCII码)	从控制台读取一个字符 [citation:5]
time	30	无	`$a0` = 系统时间低32位 `$a1` = 系统时间高32位	获取当前系统时间（毫秒级） [citation:5]
sleep	32	`$a0` = 休眠的毫秒数	无	让程序暂停执行指定的毫秒数 [citation:5]
print_hex	34	`$a0` = 要打印的整数	无	以8位十六进制形式打印整数（左侧补零） [citation:5]
print_binary	35	`$a0` = 要打印的整数	无	以32位二进制形式打印整数（左侧补零） [citation:5]

服务名称	服务代码 ($v0)	需要的参数	返回值	功能说明
print_unsigned	36	`$a0` = 要打印的整数	无	以无符号十进制形式打印整数 [citation:5]
random_int	41	`$a0` = 随机数生成器ID (通常为0)	`$a0` = 生成的伪随机整数	生成一个均匀分布的伪随机整数 [citation:5]
random_range	42	`$a0` = 随机数生成器ID (通常为0) `$a1` = 上限值	`$a0` = 生成的伪随机整数	生成一个在 `[0, 上限值)` 范围内的伪随机整数 [citation:5]
confirm_dialog	50	`$a0` = 提示消息字符串地址	`$a0` = 用户选择 (0:Yes, 1:No, 2:Cancel)	弹出带有 Yes/No/Cancel 的确认对话框 [citation:5]
input_int_dialog	51	`$a0` = 提示消息字符串地址	`$a0` = 输入的整数 `$a1` = 状态码 (0:成功, -1/-2/-3:失败)	弹出输入对话框让用户输入一个整数 [citation:5]
message_dialog	55	`$a0` = 消息字符串地址 `$a1` = 对话框类型 (0:错误, 1:信息, 2:警告, 3:问号)	无	弹出一个消息提示对话框 [citation:5]